

DESIGN PATTERNS

What are Design Patterns?

Design patterns are reusable solutions to common problems that software developers encounter in software design. They provide standard ways of organizing and structuring code to solve a specific type of problem, making the code more flexible, reusable, and maintainable.

- Design patterns are **not finished designs** but templates for solving problems.
- They describe solutions in terms of object interactions, responsibilities, and collaborations between classes or objects.

Why are Design Patterns Used?

1. **Reusable Solutions:**
 - o Design patterns provide proven solutions that have been tested and used by many developers. This prevents reinventing the wheel when solving recurring problems.
2. **Code Readability and Maintainability:**
 - o Patterns promote **clear code structure** and standard practices, which make the code easier to understand, modify, and extend by other developers.
3. **Communication:**
 - o Design patterns serve as a common vocabulary among developers. Instead of explaining a complex design from scratch, you can say “we’ll use a Singleton” or “this calls for a Strategy,” and developers will immediately understand the approach.
4. **Best Practices:**
 - o Patterns encapsulate best practices in software development. By following them, developers ensure that their designs are robust, flexible, and scalable.
5. **Encapsulation of Change:**
 - o They help encapsulate areas of the code that are likely to change. For example, the **Strategy pattern** allows developers to switch between different algorithms without altering the client code.
6. **Promote Design Principles:**
 - o Many patterns are based on solid object-oriented principles, such as:
 - **Single Responsibility Principle (SRP):** Each class should have one reason to change.
 - **Open-Closed Principle (OCP):** Software entities should be open for extension but closed for modification.
 - **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules but on abstractions.

Categories of Design Patterns:

Design patterns are often categorized into three broad types:

1. **Creational Patterns:**
 - o Focus on how objects are created. They help make the system independent of how its objects are constructed.
 - o **Examples:** Singleton, Factory, Abstract Factory, Builder, Prototype.
2. **Structural Patterns:**
 - o Deal with object composition and the relationships between objects. They help ensure that if one part of a system changes, the entire system doesn't need to change.
 - o **Examples:** Adapter, Bridge, Composite, Decorator, Facade, Proxy.
3. **Behavioral Patterns:**
 - o Concerned with how objects interact and communicate with each other.
 - o **Examples:** Observer, Strategy, Command, State, Chain of Responsibility, Mediator, Visitor.

TYPES OF DESIGN PATTERNS

Singleton Pattern

Definition:

The **Singleton Pattern** is a creational design pattern that ensures a class has **only one instance** and provides a global point of access to that instance.

In simpler terms, the Singleton Pattern restricts the instantiation of a class to a single object and makes that instance accessible throughout the application.

Characteristics of Singleton Pattern:

1. **Single Instance:** The Singleton Pattern guarantees that only one instance of the class is created, no matter how many times the class is accessed.
2. **Global Access:** The single instance is usually accessible via a static method (e.g., `get_instance()`), making it globally accessible.
3. **Private Constructor:** The constructor is made **private** to prevent direct instantiation from outside the class. The constructor of the Singleton class is declared private (or protected, depending on the programming language) so that it cannot be called from outside the class. This prevents direct instantiation using the `new` keyword or equivalent.

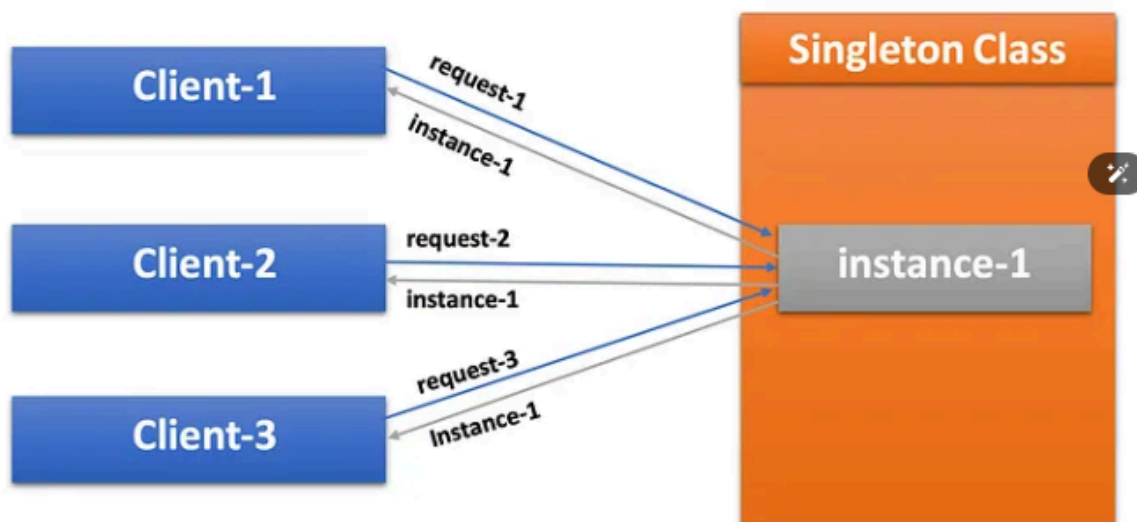
When to Use Singleton Pattern:

- When exactly **one instance** of a class is needed across the system (e.g., managing system resources).
- When you want to **control access** to a shared resource, such as a database connection, logging system, or printer spooler.
- In cases where **global state** or **shared configuration** needs to be accessible throughout the application.

Example Use Cases:

1. **Logging:** An application typically has one logger instance to avoid conflicts and ensure that all parts of the system write to the same log.
2. **Configuration Settings:** A class that loads configuration settings for an application can use a Singleton to ensure that the settings are loaded once and shared globally.
3. **Database Connections:** You may want to limit database connections to a single instance to avoid performance overhead or conflicts.

Singleton design pattern



Singleton Design Pattern

Implementation of Singleton Pattern (in Python):

Example Code:

Class Singleton:

```
_instance = None # This is a private class-level attribute to store the single instance  
  
def __init__(self):  
  
    # Preventing new instances if one already exists  
  
    if Singleton._instance is not None:  
  
        raise Exception("This class is a singleton! Use get_instance() method to access the instance.")  
  
    else:  
  
        Singleton._instance = self  
  
        # Initialize your class (e.g., setting up resources)  
  
        print("Singleton Instance Created")  
  
@staticmethod  
  
def get_instance():  
  
    if Singleton._instance is None:  
  
        Singleton()  
  
    return Singleton._instance  
  
# Usage Example:  
  
singleton1 = Singleton.get_instance()  
  
singleton2 = Singleton.get_instance()  
  
# Both variables will point to the same instance  
  
print(singleton1 is singleton2) # Output: True (Both are the same instance)
```

Explanation of the Code:

- `_instance`: This is a **class-level variable** that holds the single instance of the Singleton class.

- **Private constructor (__init__):** The constructor is designed to throw an exception if an attempt is made to create more than one instance.
- **Static method (get_instance):** This method checks if the instance already exists. If it doesn't, it creates one. If it does, it returns the existing instance.

How It Works:

1. The first time get_instance() is called, the Singleton instance is created.
2. On subsequent calls to get_instance(), the same instance is returned.
3. If someone tries to create another instance directly using the constructor (Singleton()), the system raises an exception.

Advantages of Singleton Pattern:

1. **Controlled Access to a Single Instance:** The Singleton Pattern ensures that a class has only one instance and provides a controlled access point.
2. **Saves Memory:** Since only one instance is created, the Singleton Pattern can save system resources, especially in cases where object creation is resource-intensive.
3. **Global State Management:** Singleton allows the management of shared resources such as configuration, logging, or database connections globally.

Disadvantages of Singleton Pattern:

1. **Global State Can Be Risky:** Singleton introduces global state into the application, which can lead to problems like **tight coupling** and **difficulty in testing**.
2. **Difficult to Unit Test:** Singleton classes are difficult to mock in unit tests because of their global state. Test cases can become dependent on the state of the Singleton.
3. **Breaks SOLID Principles:** Singleton often violates the **Single Responsibility Principle (SRP)** because it controls both the instance creation and business logic.

How Singleton Violates SRP

1. **Instance Creation Responsibility:**
 - In a typical Singleton implementation, the class itself is responsible for creating and managing its own instance. This means that the class handles the instantiation logic (e.g., checking if an instance already exists and creating one if it doesn't).
2. **Business Logic Responsibility:**
 - Besides managing its own instance, the Singleton class usually also contains business logic—methods and properties that define its primary functionality or purpose. This means the class has to handle two responsibilities: controlling instance creation and executing business logic.

Common Variants of Singleton:

1. **Eager Initialization:** The Singleton instance is created when the class is loaded, not on demand.
 - o **Problem:** Wastes resources if the instance is never used.
2. **Lazy Initialization:** The Singleton instance is created the first time it's accessed (as shown in the example).
 - o **Benefit:** Saves resources by creating the instance only when it's actually needed.
3. **Thread-Safe Singleton:** Ensures that in a multi-threaded environment, only one instance of the Singleton class is created.
 - o **Solution:** Use thread locks or synchronization to ensure thread safety.

Real-Life Example of Singleton: Printer Spooler

A **printer spooler** should be a Singleton because:

- Only one spooler should manage the print jobs.
- Multiple spoolers would cause confusion and inconsistency when printing documents.

In the printer system:

- The **Printer Spooler** class manages all print requests.
- Only one instance of the spooler exists to ensure that all print jobs are queued correctly.

2. Factory Pattern

Definition:

The **Factory Pattern** is a **creational design pattern** that provides an interface for creating objects in a superclass, but allows subclasses to **alter the type of objects** that will be created.

In simpler terms, the Factory Pattern lets you define a method for creating objects, but lets the child classes decide which class to instantiate. It helps in delegating the process of object creation and promotes **loose coupling** between the code that uses objects and the code that creates them.

Key Components of the Factory Pattern:

1. Product (Interface/Abstract Class):

- o This is the interface or abstract class that defines the type of object the factory will create. An **abstract class** is a fundamental concept in object-oriented programming (OOP) that serves as a blueprint for other classes. It cannot be instantiated on its own, meaning you cannot create an object directly from an abstract class. Instead, it is meant to be subclassed, and its concrete implementations are provided by derived classes

2. Concrete Products:

- o These are the actual implementations or subclasses that will be created by the factory method.

3. Creator (Factory):

- o The class that contains the method (createProduct()) to create objects.
- o This class does not directly instantiate objects but delegates the responsibility of object creation to subclasses.

4. Concrete Creators:

- o These are the subclasses that **override** the factory method to decide which specific product to create.
-

When to Use the Factory Pattern:

- When you need to create objects but don't know in advance the exact class of the objects that will be created.
 - When the creation logic of objects is complex or involves multiple steps.
 - When you want to **encapsulate** object creation logic and centralize it to make it easier to manage changes or extend.
-

Example Use Cases:

1. **Document Creation:** A text editor can create multiple types of documents (e.g., Word, PDF, HTML). Instead of directly creating these objects, a factory method can be used to determine which document type to create.

2. **Shape Creation:** A drawing application can have different shapes (e.g., Circle, Square, Rectangle). The factory pattern can be used to create different shapes dynamically without knowing which one will be needed at runtime.

Implementation of Factory Pattern (in Python):

Example: A Shape Factory

Step 1: Define the Product Interface or Abstract Class

```
class Shape:
```

```
    def draw(self):
```

```
        pass
```

Step 2: Create Concrete Products (Specific implementations of Shape)

```
class Circle(Shape):
```

```
    def draw(self):
```

```
        return "Drawing a Circle"
```

```
class Square(Shape):
```

```
    def draw(self):
```

```
        return "Drawing a Square"
```

```
class Rectangle(Shape):
```

```
    def draw(self):
```

```
        return "Drawing a Rectangle"
```

Step 3: Create the Factory (Creator) Class

```
class ShapeFactory:
```

```
    @staticmethod
```

```
    def get_shape(shape_type):
```

```
        # Step 4: Implement the Factory Method
```

```
        if shape_type == "CIRCLE":
```

```
            return Circle()
```

```
        elif shape_type == "SQUARE":
```

```
            return Square()
```

```
        elif shape_type == "RECTANGLE":
```

```
            return Rectangle()
```

```
        else:
```

```
            return None
```

```
# Usage Example:
```

```
shape_factory = ShapeFactory()
```

```
# Creating and using shapes
```

```
shape1 = shape_factory.get_shape("CIRCLE")
```

```
print(shape1.draw()) # Output: Drawing a Circle
```

```
shape2 = shape_factory.get_shape("SQUARE")
```

```
print(shape2.draw()) # Output: Drawing a Square
```

```
shape3 = shape_factory.get_shape("RECTANGLE")
```

```
print(shape3.draw()) # Output: Drawing a Rectangle
```

Explanation of the Code:

1. **Product Interface (Shape):** This is an abstract class that defines a method `draw()` which all concrete shapes must implement.
2. **Concrete Products (Circle, Square, Rectangle):** These classes implement the Shape interface and define how each specific shape is drawn.
3. **Factory Class (ShapeFactory):** This class contains the `get_shape()` method, which is the factory method responsible for creating different shapes based on the input.
4. **Usage:** The client code (in this case, the user) requests a specific shape by passing the shape type to the factory (`get_shape()` method), and the factory returns an instance of the appropriate shape.

Advantages of Factory Pattern:

1. **Encapsulation of Object Creation:**
 - o The factory encapsulates the object creation logic, which means the client code doesn't need to know the details of how objects are created or which specific class to instantiate.
2. **Loose Coupling:**
 - o The client is decoupled from the specific classes it needs. The factory pattern allows the code to depend on abstractions (interfaces) rather than concrete implementations.
3. **Flexibility:**
 - o It becomes easier to add new types of objects (new shapes in the example) without changing the client code. You just need to add a new product and update the factory.
4. **Single Responsibility Principle (SRP):**
 - o The Factory Pattern promotes SRP by separating the responsibility of object creation into its own class.

Disadvantages of Factory Pattern:

1. **Complexity for Simple Applications:**

- o For small applications where you don't have multiple types of objects to create, using a factory pattern may introduce unnecessary complexity.
2. **Additional Overhead:**
- o If the number of product types grows, the factory method logic can become cumbersome to maintain as it may require a large if-else or switch statement to decide which object to create.

Variations of Factory Pattern:

1. **Simple Factory:**
 - o A simple implementation of the factory pattern where a factory method is responsible for creating all types of objects. This is closer to the example provided above.
2. **Factory Method Pattern:**
 - o In this version, the factory method is defined in an abstract class, and each subclass provides its own implementation of the factory method. This allows for more flexibility in object creation.
 - o **Example:** Different document types (PDF, Word, Excel) can have different factory classes that extend a common abstract class.
3. **Abstract Factory:**
 - o The **Abstract Factory** Pattern is an extension of the Factory Method Pattern. It is used when there is a need to create families of related or dependent objects.
 - o **Example:** A furniture factory that can create related products like chairs, tables, and sofas from different product families (e.g., Modern, Victorian).

Real-Life Example of Factory Pattern: Vehicle Factory

Imagine an application where users can choose to rent different types of vehicles: a car, a truck, or a bike. Instead of directly creating objects of these vehicle types, a **Vehicle Factory** can be used to dynamically create objects based on the user's selection.

Vehicle Factory Example:

class Vehicle:

def rent(self):

pass

class Car(Vehicle):

```
def rent(self):  
    return "Renting a Car"
```

```
class Truck(Vehicle):  
    def rent(self):  
        return "Renting a Truck"
```

```
class Bike(Vehicle):  
    def rent(self):  
        return "Renting a Bike"
```

```
class VehicleFactory:  
    @staticmethod  
    def get_vehicle(vehicle_type):  
        if vehicle_type == "CAR":  
            return Car()  
        elif vehicle_type == "TRUCK":  
            return Truck()  
        elif vehicle_type == "BIKE":  
            return Bike()  
        else:  
            return None
```

Usage:

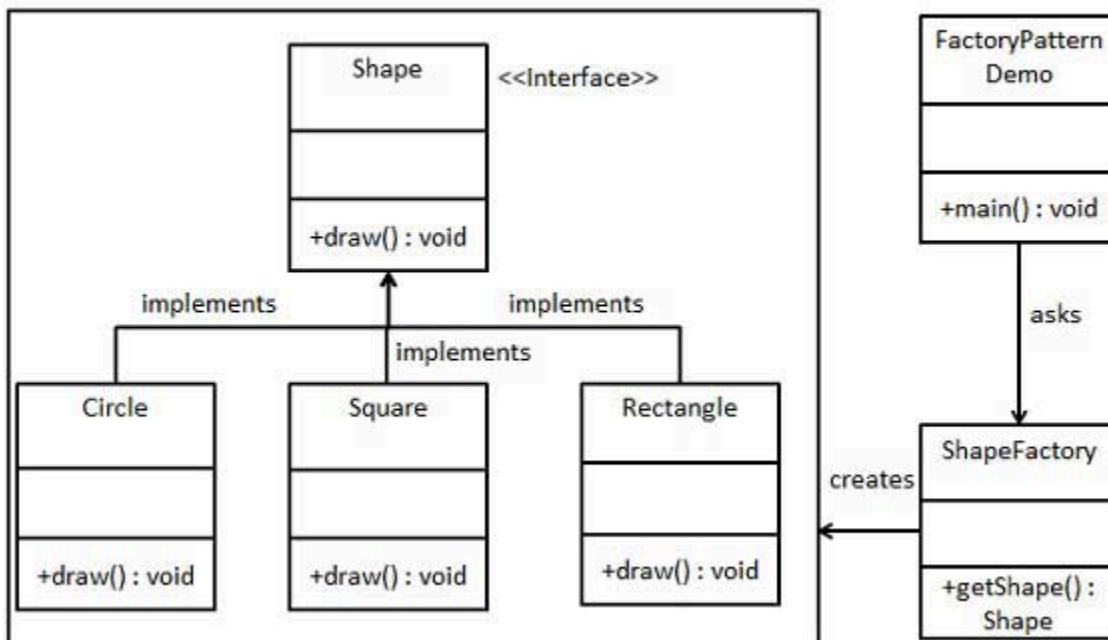
```
vehicle_factory = VehicleFactory()
```

```
vehicle = vehicle_factory.get_vehicle("CAR")
```

```
print(vehicle.rent()) # Output: Renting a Car
```

In this case, the client (user) does not need to know how the car, truck, or bike objects are created. They simply ask the factory to give them the right vehicle based on the input.

ShapeFactory to get the type of object it needs.



3. Observer Pattern

Observer Design Pattern

Definition:

The **Observer Pattern** is a behavioral design pattern in which an object, known as the **subject**, maintains a list of its dependents, called **observers**, and notifies them of any state changes, typically by calling one of their methods. This pattern is used to **implement distributed event handling** systems, where multiple objects need to be informed of changes in another object.

It essentially defines a **one-to-many relationship** between objects so that when the state of one object changes, all its dependents (observers) are notified and updated automatically.

Key Components of the Observer Pattern:

1. **Subject:**

- o The object that holds the state and notifies observers of changes.
- o The subject provides methods for adding or removing observers and for notifying them when the state changes.

2. **Observers:**

- o The objects that are interested in being notified about changes in the subject's state.
- o Observers register with the subject to receive updates and implement an update method that is called when the subject changes.

3. **Concrete Subject:**

- o A specific implementation of the subject that can be observed. When its state changes, it notifies the observers.

4. **Concrete Observers:**

- o The specific implementations of observers that take some action when they receive notifications from the subject.

When to Use the Observer Pattern:

- When an object's state changes and **multiple objects** need to be updated automatically.
- When you want to implement **event-driven systems** where an action in one part of the system triggers actions in other parts.
- When the number of objects interested in an event can change dynamically (i.e., objects can subscribe or unsubscribe at runtime).

Example Use Cases:

1. **Graphical User Interfaces (GUI):** In GUI frameworks, different components (observers) are updated when the user interacts with another component (subject), such as changing a value in a dropdown or input field.
2. **Notifications Systems:** Applications that send notifications to users when specific events occur (e.g., when a product price drops, or when new content is published).
3. **Event Management:** Logging systems that record events whenever certain actions occur in a program.

Implementation of Observer Pattern (in Python):

Example: Weather Station (Subject) and Display Devices (Observers)

Step 1: Define the Observer Interface

class Observer:

```
def update(self, temperature, humidity):  
  
    pass
```

Step 2: Define the Subject (Observable) Class

```
class WeatherStation:
```

```
    def __init__(self):  
  
        self._observers = []  
  
        self._temperature = None  
  
        self._humidity = None
```

```
    def add_observer(self, observer):  
  
        self._observers.append(observer)
```

```
    def remove_observer(self, observer):  
  
        self._observers.remove(observer)
```

```
    def notify_observers(self):  
  
        for observer in self._observers:  
  
            observer.update(self._temperature, self._humidity)
```

```
    def set_measurements(self, temperature, humidity):  
  
        self._temperature = temperature  
  
        self._humidity = humidity
```

```
self.notify_observers()
```

Step 3: Define Concrete Observers

```
class PhoneDisplay(Observer):
```

```
    def update(self, temperature, humidity):
```

```
        print(f"Phone Display: Temperature is {temperature}°C and humidity is {humidity}%")
```

```
class LaptopDisplay(Observer):
```

```
    def update(self, temperature, humidity):
```

```
        print(f"Laptop Display: Temperature is {temperature}°C and humidity is {humidity}%")
```

Usage Example

```
weather_station = WeatherStation()
```

Create observers

```
phone_display = PhoneDisplay()
```

```
laptop_display = LaptopDisplay()
```

Register observers with the weather station

```
weather_station.add_observer(phone_display)
```

```
weather_station.add_observer(laptop_display)
```

Simulate changes in weather conditions

weather_station.set_measurements(25, 60)

Output:

Phone Display: Temperature is 25°C and humidity is 60%

Laptop Display: Temperature is 25°C and humidity is 60%

weather_station.set_measurements(30, 50)

Output:

Phone Display: Temperature is 30°C and humidity is 50%

Laptop Display: Temperature is 30°C and humidity is 50%

Explanation of the Code:

1. **Observer Interface (Observer):** This defines the `update()` method, which will be called by the subject to notify the observer of state changes.
2. **Concrete Observers (PhoneDisplay, LaptopDisplay):** These classes implement the `update()` method to receive notifications and take appropriate actions when the state of the weather station changes.
3. **Subject (WeatherStation):** The weather station keeps a list of observers. When the weather data (temperature and humidity) changes, it notifies all registered observers by calling their `update()` methods.

How It Works:

- The **weather station** is the subject, and the **phone display** and **laptop display** are observers.
- When the weather station updates its data (temperature and humidity), it automatically **notifies all registered observers**, which then update themselves with the new data.

Advantages of Observer Pattern:

1. **Loose Coupling:**
 - o The subject and observers are loosely coupled, meaning the subject doesn't need to know about the concrete classes of the observers. It only knows that they implement a certain interface (in this case, `Observer`).

- This makes the system more flexible and easier to maintain, as you can add or remove observers without changing the subject.
- 2. **Dynamic System:**
 - Observers can be added or removed dynamically at runtime, offering flexibility in how notifications are managed.
- 3. **Code Reusability:**
 - The same subject can be used with different types of observers without needing to modify the subject's code. Each observer can handle notifications in its own way.
- 4. **Scalability:**
 - The pattern can easily handle an increasing number of observers, making it suitable for systems that need to broadcast events to many listeners.

Disadvantages of Observer Pattern:

1. **Memory Leaks:**
 - If observers are not properly removed (unsubscribed) from the subject, this can lead to memory leaks, especially in environments like GUIs where observers (e.g., UI components) may need to be removed dynamically.
2. **Unexpected Updates:**
 - If not carefully designed, the subject can update observers unexpectedly, leading to unintended side effects or performance issues if there are many observers.
3. **Broadcast Overhead:**
 - In systems with many observers, notifying all of them can cause performance overhead, especially if the notifications occur frequently or the observers perform complex tasks in response to updates.

Variations of the Observer Pattern:

1. **Push vs Pull Model:**
 - **Push Model:** The subject sends **all data** to observers in the notification, whether or not they need it. This is simpler but less efficient.
 - **Pull Model:** The subject sends a **minimal notification**, and observers can choose to query the subject for the data they need. This is more efficient but requires more coordination between the subject and observers.
2. **Event-driven Architecture:**
 - In many systems, the Observer Pattern is implemented through event listeners and handlers, where objects subscribe to events and get notified when the event occurs.
3. **Multicast Observer:**

- o The observer pattern can be extended to allow **broadcasting notifications to a subset** of observers based on certain criteria (e.g., priority or type of notification).

Real-Life Example of Observer Pattern: Newsletter Subscription

Imagine an online store where users can subscribe to receive notifications about new products or sales:

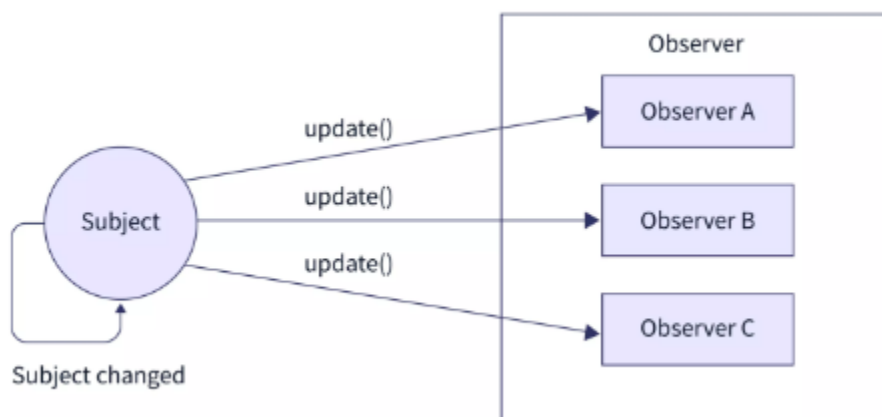
- The **subject** is the online store.
- The **observers** are the customers who subscribe to the newsletter.
- When the store has a new product or sale, it **notifies** all subscribed customers by sending them an email or message.

In this case:

- Customers (observers) can **subscribe** or **unsubscribe** from the notifications (add or remove themselves from the subject).
- The online store (subject) does not need to know any details about the customers (observers) but can notify all of them when there is an update.

Summary:

- The **Observer Pattern** defines a **one-to-many relationship** between objects so that when the state of one object (subject) changes, all dependent objects (observers) are notified automatically.
- It is commonly used in **event-driven systems** and **distributed applications** to handle updates between objects.
- The pattern promotes **loose coupling** and allows systems to be dynamically updated by adding or removing observers at runtime. However, it can introduce complexity if not carefully managed, especially with respect to memory leaks and performance.



4. Strategy Pattern

Definition:

The **Strategy Pattern** is a **behavioral design pattern** that enables selecting an algorithm's behavior at runtime. It defines a family of algorithms, encapsulates each one, and makes them interchangeable. This pattern allows the algorithm to vary independently from the clients that use it.

Simply put, the Strategy Pattern allows a class to **change its behavior dynamically** by selecting one of many interchangeable strategies (algorithms) at runtime.

Key Components of the Strategy Pattern:

1. **Context:**
 - o The class that contains a reference to a strategy. The context interacts with a strategy object and uses it to perform a task.
 2. **Strategy Interface:**
 - o This defines a common interface for all strategies (algorithms) so that the context can interact with them interchangeably.
 3. **Concrete Strategies:**
 - o These are the specific implementations of the strategy interface. Each concrete strategy implements a different algorithm that the context can use.
-

When to Use the Strategy Pattern:

- When you have multiple ways to perform a specific task and want to allow the client to choose the algorithm at runtime.
 - When you want to avoid using large conditionals (like if-else or switch statements) to select different behaviors.
 - When you want to decouple the algorithm from the class that uses it, making it easier to extend or change algorithms independently.
-

Example Use Cases:

1. **Sorting Algorithms:** A class that needs to sort data might choose different sorting strategies (e.g., Bubble Sort, Quick Sort, Merge Sort) depending on the size of the data set or other factors.
2. **Payment Processing:** An online store might use different payment strategies (e.g., Credit Card, PayPal, Bitcoin) depending on the user's choice.

3. **Compression Algorithms:** A file compression tool might choose different compression strategies (e.g., ZIP, RAR, TAR) depending on the file type or user preference.

Implementation of Strategy Pattern (in Python):

Example: Payment Processing System

Step 1: Define the Strategy Interface

```
class PaymentStrategy:
```

```
    def pay(self, amount):
```

```
        pass
```

Step 2: Define Concrete Strategies (Different payment methods)

```
class CreditCardPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f"Paid {amount} using Credit Card."
```

```
class PayPalPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f"Paid {amount} using PayPal."
```

```
class BitcoinPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f"Paid {amount} using Bitcoin."
```

Step 3: Define the Context (which uses a payment strategy)

```
class PaymentContext:

    def __init__(self, strategy: PaymentStrategy):

        self._strategy = strategy

    def set_strategy(self, strategy: PaymentStrategy):

        self._strategy = strategy

    def pay_amount(self, amount):

        return self._strategy.pay(amount)

# Usage Example

payment_context = PaymentContext(CreditCardPayment())

print(payment_context.pay_amount(100)) # Output: Paid 100 using Credit Card.

# Change strategy to PayPal

payment_context.set_strategy(PayPalPayment())

print(payment_context.pay_amount(200)) # Output: Paid 200 using PayPal.

# Change strategy to Bitcoin

payment_context.set_strategy(BitcoinPayment())

print(payment_context.pay_amount(300)) # Output: Paid 300 using Bitcoin.
```

Explanation of the Code:

1. **Strategy Interface (PaymentStrategy):** This defines the common interface `pay()` that all payment methods must implement.
2. **Concrete Strategies (CreditCardPayment, PayPalPayment, BitcoinPayment):** These are the specific implementations of the `PaymentStrategy` interface. Each class provides a different way to process the payment.
3. **Context (PaymentContext):** The context holds a reference to a strategy object and uses it to process payments. The client can change the strategy at runtime by calling `set_strategy()`.

How It Works:

- The client (user) interacts with the **PaymentContext** and can choose different payment methods (strategies) dynamically.
- The **PaymentContext** delegates the payment processing to the selected strategy without needing to know how each payment method is implemented.

Advantages of Strategy Pattern:

1. **Decouples Algorithm from Context:**
 - The strategy pattern allows the algorithm (strategy) to be separated from the context (the class that uses it). This makes the context more flexible, as it can use different strategies without needing to modify its code.
2. **Open/Closed Principle:**
 - The strategy pattern makes it easy to add new strategies (algorithms) without changing the existing context or strategy interface. You simply add a new strategy class, making the system open for extension but closed for modification.
3. **Avoids Conditional Statements:**
 - Instead of using large if-else or switch statements to choose between different behaviors, the strategy pattern uses polymorphism to achieve the same result in a cleaner and more maintainable way.
4. **Runtime Flexibility:**
 - The pattern allows the client to change the algorithm at runtime, providing dynamic flexibility for applications that need to change behavior based on different conditions.

Disadvantages of Strategy Pattern:

1. **Increased Number of Classes:**
 - For each new strategy, you need to create a new class. This can increase the number of classes in your system, potentially leading to more complexity in terms of class management.
2. **Clients Must Know About Different Strategies:**

- o The client (context) must be aware of the different strategies available and know when to use which one. This requires the client to understand the pros and cons of each strategy.
3. **Overhead for Simple Cases:**
- o If there are only a few strategies and they don't vary much, using the strategy pattern might introduce unnecessary complexity. Simple if-else statements might be sufficient in such cases.
-

Variations of Strategy Pattern:

1. **Context-Specific Strategies:**
 - o In some cases, strategies may be defined for specific contexts. For instance, if a particular strategy only applies under certain conditions, the context might be responsible for switching strategies dynamically.
 2. **Strategy Caching:**
 - o Some implementations may cache strategies for reuse instead of creating new instances of strategy objects every time, improving performance in situations where strategies are frequently switched.
 3. **Lambda Expressions (in Modern Languages):**
 - o In languages like Python or JavaScript, you can use functions or lambda expressions to implement strategies, reducing the need for explicit strategy classes.
-

Real-Life Example of Strategy Pattern: Travel Booking

Imagine a travel booking application where a customer can choose different travel strategies based on time and cost preferences:

- **By Car:** Cheaper but takes longer.
- **By Plane:** Faster but more expensive.
- **By Train:** A balance between time and cost.

The user (context) can select a travel strategy at runtime, and the booking system will execute the chosen travel method. The strategy pattern would allow the booking system to dynamically switch between these options based on user input.

Summary:

- The **Strategy Pattern** defines a family of algorithms (strategies) and makes them interchangeable, allowing clients to select an algorithm at runtime.

- It promotes **loose coupling** by separating the algorithm from the context, making it easier to add or change algorithms without modifying the context's code.
- The strategy pattern is useful for systems where you have multiple ways of doing something, and you want to dynamically select the best option based on user input or runtime conditions.

Pattern	Intent	Key Features	Use Case
Singleton	Ensure only one instance of a class exists	Private constructor, global access method	Global access to logging, config
Factory	Provide an interface for creating objects without specifying the exact type	Decouples object creation, promotes extensibility	Runtime selection of object types
Observer	Define a one-to-many dependency between objects	Subject and observer, automatic notifications	Event systems, GUIs
Strategy	Encapsulate interchangeable algorithms or behaviors	Different strategies encapsulated in classes, selected dynamically	Sorting algorithms, payment methods